

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский авиационный институт (национальный исследовательский  
университет)» (МАИ)

Институт № 3 — «Системы управления, информатики и электроэнергетика»

Кафедра 307 — «Цифровые технологии и информационные системы»

## **ОТЧЁТ**

### **О ПРОХОЖДЕНИИ УЧЕБНОЙ ПРАКТИКИ**

по теме:

### **РАЗРАБОТКА ГЕНЕРАТОРА DXF-ЧЕРТЕЖЕЙ ДЛЯ ЛАЗЕРНОЙ РЕЗКИ**

Выполнил: студент  
группы МЗО-113БВ-25

Садунян Р.К.

Принял: старший  
преподаватель

подпись, дата

Павлов О.В.

подпись, дата

Москва 2026

## СОДЕРЖАНИЕ

|                                                          |    |
|----------------------------------------------------------|----|
| Введение .....                                           | 3  |
| 1 Описание изделия .....                                 | 4  |
| 2 Описание программы .....                               | 6  |
| 2.1 Ввод и проверка параметров .....                     | 6  |
| 2.2 Хранение геометрии .....                             | 7  |
| 3 Алгоритмы построения деталей .....                     | 8  |
| 3.1 Расположение шипов .....                             | 8  |
| 3.2 Построение стенок .....                              | 8  |
| 3.3 Основание .....                                      | 9  |
| 3.4 Крышка .....                                         | 9  |
| 3.5 Монтажные уголки .....                               | 9  |
| 3.6 Создание комплекта и раскладка .....                 | 9  |
| 4 Создание DXF-чертежа .....                             | 11 |
| 4.1 Проверка геометрии .....                             | 13 |
| 5 Изготовление изделия .....                             | 14 |
| Заключение .....                                         | 15 |
| Список использованных источников .....                   | 16 |
| Приложение А Исходный код программы .....                | 17 |
| Приложение А.1 Точка входа и графический интерфейс ..... | 17 |
| Приложение А.2 Расчёт геометрии .....                    | 20 |
| Приложение А.3 Экспорт DXF .....                         | 35 |

## **ВВЕДЕНИЕ**

Во время практики была разработана программа для автоматического создания DXF-чертежа коробки ElectronicsBox. Пользователь вводит размеры будущего изделия, после чего программа рассчитывает все детали и сохраняет готовый файл для лазерной резки.

Разрабатываемый DXF должен содержать согласованные шипы и пазы, непересекающиеся детали раскладки и замкнутые контуры реза. Корректность результата проверена построением трёхмерной модели и изготовлением изделия из фанеры.

## 1 Описание изделия

ElectronicsBox представляет собой закрытую коробчатую конструкцию для размещения электронных компонентов.

Изделие состоит из следующих плоских деталей:

- крышки;
- основания (дна) с закруглёнными углами;
- четырёх стенок;
- четырёх монтажных уголков.

Стенки соединяются между собой прямоугольными шипами и пазами. Основание выступает за пределы стенок и крепится к ним пазовыми соединениями. Крышка фиксируется сверху. Монтажные уголки связывают стенки и крышку, обеспечивая жёсткость конструкции.

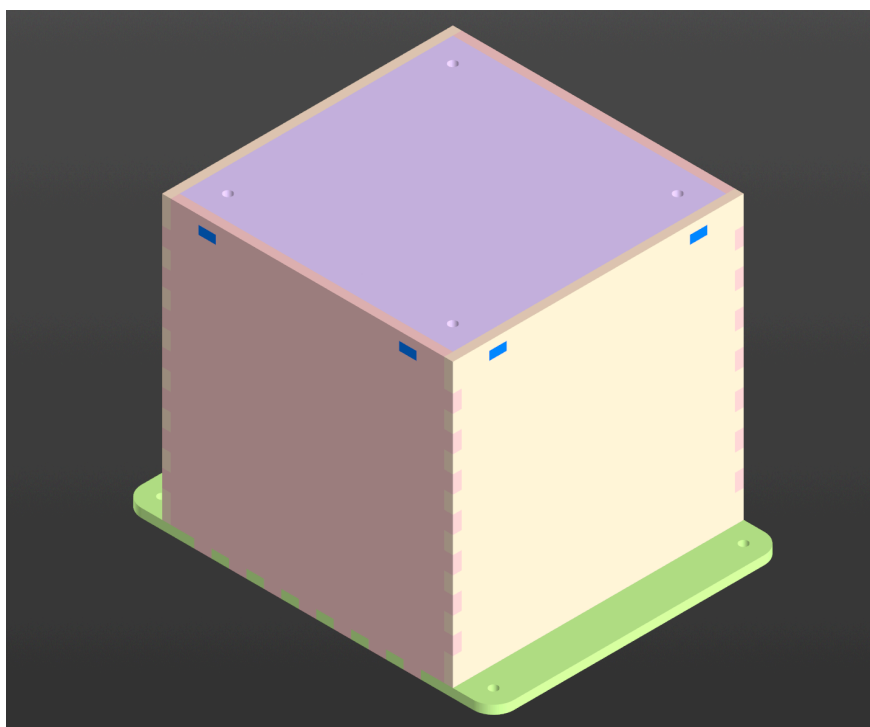


Рисунок 1 — Трёхмерная модель сборки ElectronicsBox

Пользователь может изменять параметры, приведённые в таблице 1.

Таблица 1 — Основные параметры программы

| Параметр                     | По умолчанию        | Назначение                     |
|------------------------------|---------------------|--------------------------------|
| Длина (внутренняя ширина)    | 100 мм              | Размер изделия слева направо   |
| Глубина (внутренняя глубина) | 100 мм              | Размер изделия спереди назад   |
| Высота (внутренняя высота)   | 100 мм              | Полезная высота изделия        |
| Толщина материала            | 3 мм                | Глубина шипов и размер пазов   |
| Ширина шипа                  | 6 мм                | Желаемый размер соединений     |
| Имя выходного файла          | electronics_box.dxf | Путь для сохранения результата |

## 2 Описание программы

Программа реализована на Python с использованием tkinter для графического интерфейса и ezdxf для формирования DXF.

Функциональные модули проекта:

- `laser_box_generator.py` — графический интерфейс и точка входа;
- `box_geometry.py` — расчёт геометрии деталей, валидация и раскладка;
- `dxf_writer.py` — формирование и сохранение DXF.

Интерфейс получает значения от пользователя, а геометрические расчёты выполняются в отдельном модуле.



Рисунок 2 — Этапы создания DXF-чертежа

### 2.1 Ввод и проверка параметров

После запуска отображается окно с полями параметров и кнопкой «Сгенерировать DXF». Поля заполнены значениями по умолчанию.

ElectronicsBox — генератор DXF

**Коробка ElectronicsBox**

|                                  |                     |
|----------------------------------|---------------------|
| Длина (внутренняя ширина), мм    | 100.0               |
| Глубина (внутренняя глубина), мм | 100.0               |
| Высота (внутренняя высота), мм   | 100.0               |
| Толщина материала, мм            | 3.0                 |
| Ширина шипа, мм                  | 6.0                 |
| Имя выходного файла              | electronics_box.dxf |

Сгенерировать DXF

Рисунок 3 — Интерфейс программы

Метод `get_params()` получает строки из элементов интерфейса и создаёт объект `BoxParams`. Метод `read_number()` заменяет десятичную запятую точкой

и преобразует строку в число. При ошибке преобразования выводится соответствующее сообщение.

## **2.2 Хранение геометрии**

Каждая геометрическая линия представлена объектом `Segment`, содержащим начальную и конечную точки, а также имя слоя (CUT или HOLES). Несколько сегментов группируются в объект `Panel`, содержащий заголовок и координаты размещения.

Модуль `box_geometry.py` формирует контуры деталей в виде упорядоченных списков точек, которые затем преобразуются в сегменты функцией `contours_to_segments()`. Модуль `dxwriter.py` записывает полученные сегменты как линии DXF.

Функция `clean_point()` округляет координаты до шести знаков и заменяет значения, близкие к нулю, точным нулём. Это исключает артефакты вычислений с плавающей точкой.

## 3 Алгоритмы построения деталей

### 3.1 Расположение шипов

Функция `joint_count()` рассчитывает количество шиповых соединений по длине края. Полезная длина (за вычетом толщины стенок с двух сторон) делится на удвоенную ширину шипа, что определяет количество чередующихся шипов и промежутков.

Функция `top_finger_start()` вычисляет начальное смещение первого шипа, центрируя группу шипов по длине края. Функция `side_finger_offsets()` возвращает список смещений для боковых шиповых соединений.

Сопрягаемые детали используют общие функции расчёта: на одной детали формируются выступы, на другой — соответствующие вырезы. Это обеспечивает совпадение соединений.

### 3.2 Построение стенок

Функция `wall_outline()` строит внешний контур стенки. Верхний край содержит шипы для крышки, а боковые стороны — вырезы или выступы в зависимости от типа стенки (`outer` или `inner`).

Тип `outer` применяется для передней и задней стенок: их боковые края содержат наружные шипы. Тип `inner` — для боковых стенок: их боковые края содержат ответные вырезы. Один и тот же алгоритм обхода контура используется для всех четырёх стенок.

Функции `wall_side_down()` и `wall_side_up()` формируют вершины шипов при обходе боковых сторон стенки сверху вниз и снизу вверх соответственно.

Функция `wall_contours()` дополняет внешний контур прямоугольными пазами для монтажных уголков. Пазы создаются как отдельные замкнутые контуры на слое `H0LES`.



### 3.3 Основание

Функция `rounded_bottom_outline()` формирует внешний контур основания с закруглёнными углами. На верхней и нижней сторонах (соответствующих передней и задней стенкам) располагаются пазы для шипов стенок.

Боковые стороны основания не содержат шипов — они гладкие и закруглены дугами. Функция `arc_points()` вычисляет точки дуги по центру, радиусу и начальному/конечному углам.

Основание шире стенок: оно выступает по бокам на величину, рассчитанную функцией `bottom_side_extension()`. Прямоугольные пазы по бокам основания (`bottom_slot_ys()`) предназначены для крепления к боковым стенкам. Монтажные отверстия в углах основания создаются функцией `circle_contour()`.

### 3.4 Крышка

Функция `top_contours()` формирует простой прямоугольный контур крышки без шипов. В углах крышки создаются круглые монтажные отверстия для крепления к стенкам.

### 3.5 Монтажные уголки

Функции `upper_mount_contour()` и `lower_mount_contour()` создают контуры Г-образных монтажных уголков. Каждый уголок имеет вырезы для шипов, входящих в пазы стенок, и круглое отверстие для крепежа.

Функция `mounts_contours()` создаёт четыре монтажных уголка (два верхних и два нижних), размещая их рядом друг с другом. Размер уголков масштабируется функцией `mount_size()` пропорционально наименьшему габариту коробки.

### 3.6 Создание комплекта и раскладка

Функция `build_generated_panels()` создаёт полный комплект деталей. Для каждой из четырёх стенок вычисляется положение на листе функцией

`wall_layout()`, для основания — `bottom_layout()`, для крышки — `top_layout()`, для монтажных уголков — `mounts_layout()`.

Все детали размещаются с отступом от края листа (`layout_margin()`) и промежутком между собой (`layout_gap()`). Промежуток равен половине толщины материала.

Полный порядок расчёта выглядит так:

1. берутся размеры из объекта `VoxParams`;
2. проверяются ограничения функцией `validate_params()`;
3. для каждой стенки рассчитывается положение и контуры;
4. строятся контуры основания с закруглёнными углами и пазами;
5. создаётся контур крышки с монтажными отверстиями;
6. формируются четыре монтажных уголка;
7. контуры преобразуются в сегменты и упаковываются в объект `Panel`;
8. готовые сегменты передаются в DXF-экспортёр.

## 4 Создание DXF-чертежа

Модуль `dxw_writer.py` создаёт документ DXF версии R2018 и задаёт миллиметры в качестве единиц измерения. Режущие линии помещаются на слой CUT, внутренние отверстия и пазы — на слой HOLES.

Каждый сегмент записывается командой `add_line()` с атрибутами слоя, цвета и типа линии. Заголовок документа содержит габариты чертежа (`$EXTMIN`, `$EXTMAX`).

На стандартных параметрах ( $100 \times 100 \times 100$  мм, толщина 3 мм, шип 6 мм) раскладка занимает  $201,5 \times 398,5$  мм и содержит 2864 LINE-сегмента: 664 в слое CUT и 2200 в слое HOLES.

Функция `create_dxf_document()` создаёт документ, добавляет слои функцией `add_layers()`, вычисляет все сегменты и записывает их в модельное пространство. Функция `save_dxf_file()` создаёт выходной каталог и сохраняет документ.

Метод `generate_dxf()` считывает параметры, проверяет сборку функцией `verify_assembly()`, формирует путь в каталоге `output/` и вызывает экспорт. Ошибки ввода или сохранения обрабатываются диалоговым окном.

Функция `main()` создаёт экземпляр `GeneratorApp` и запускает цикл обработки событий `tkinter`.

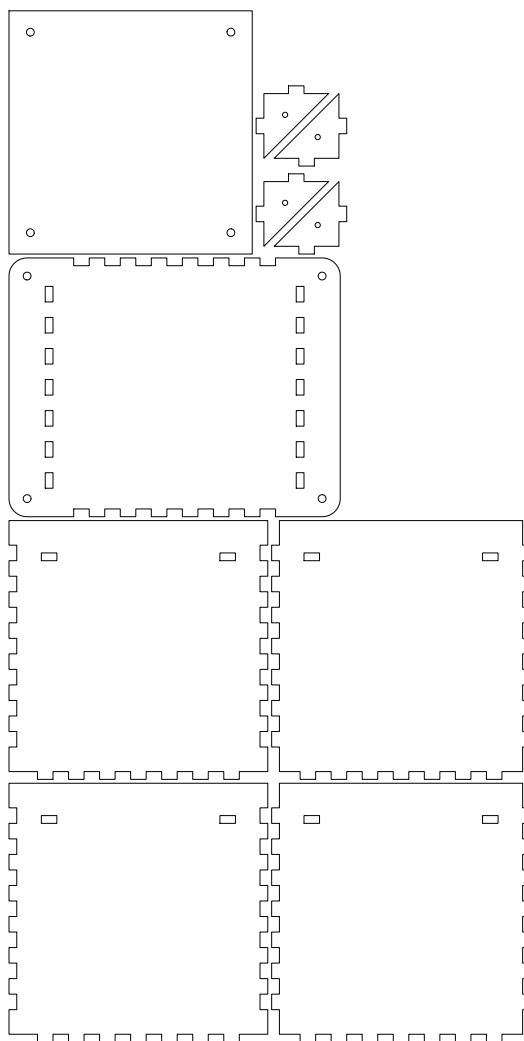


Рисунок 4 — SVG-визуализация итогового комплекта деталей

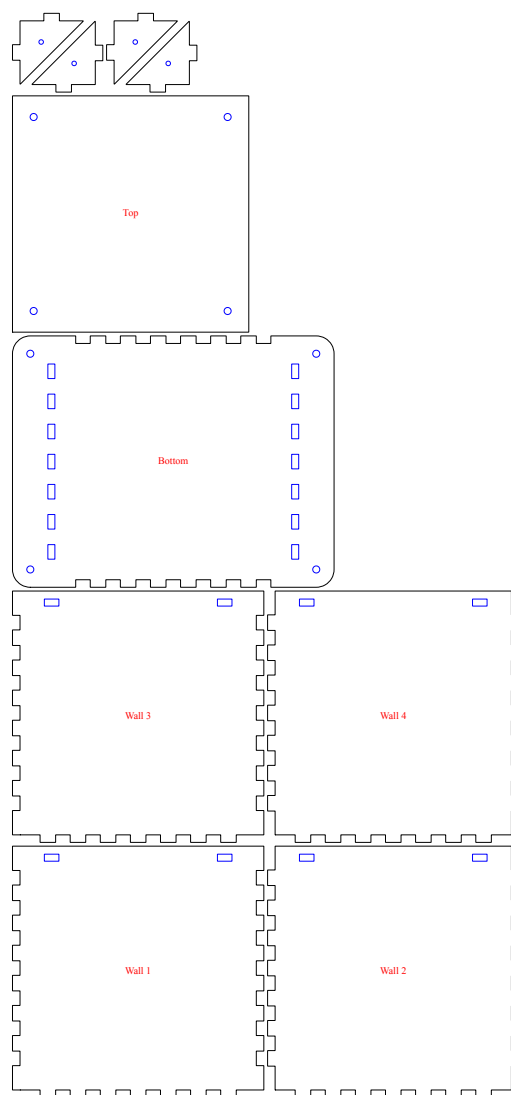


Рисунок 5 — Исходная SVG-раскладка ElectronicsBox для сравнения

#### 4.1 Проверка геометрии

Функция `verify_assembly()` выполняет автоматическую проверку перед сохранением: валидирует параметры, строит все панели и проверяет, что чертёж не выходит в отрицательные координаты и помещается на заданный размер листа.

Для визуальной проверки контуры были импортированы в КОМПАС-3D, выдавлены на толщину материала и собраны в трёхмерную модель. Проверка подтвердила взаимное положение соединений до выполнения резки.

## 5 Изготовление изделия

Для резки использовалась фанера толщиной 3 мм. В программе были введены подходящие размеры изделия и фактическая толщина материала.

Процесс изготовления был следующим:

1. программа создала DXF-чертёж;
2. файл загрузили в программу лазерного станка;
3. проверили масштаб и положение деталей на заготовке;
4. лазер вырезал все внешние контуры и отверстия;
5. готовые детали извлекли из фанеры;
6. шипы установили в соответствующие пазы и собрали изделие.

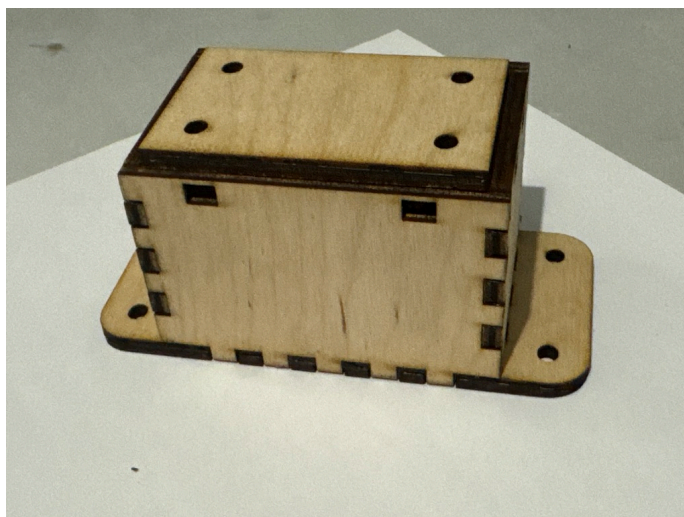


Рисунок 6 — Готовое изделие после лазерной резки и сборки

## **ЗАКЛЮЧЕНИЕ**

В результате практики была создана рабочая программа для генерации DXF-чертежа коробки ElectronicsBox. Она позволяет изменять основные размеры, автоматически рассчитывает шипы и пазы и сохраняет комплект деталей в одном файле.

Геометрия была проверена в КОМПАС-3D и реальным изготовлением. Детали успешно вырезали из фанеры и собрали, что подтвердило правильность расчётов.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Python 3 Documentation : сайт / Python Software Foundation. — URL: <https://docs.python.org/3/> (дата обращения: 06.07.2026).
2. ezdxfr documentation : сайт. — URL: <https://ezdxfr.readthedocs.io/en/stable/> (дата обращения: 06.07.2026).
3. tkinter — Python interface to Tcl/Tk : сайт / Python Software Foundation. — URL: <https://docs.python.org/3/library/tkinter.html> (дата обращения: 06.07.2026).
4. ElectronicsBox / box.LaserBiz.ru : сайт. — URL: <https://box.laserbiz.ru/site.wsgi/ElectronicsBox> (дата обращения: 07.07.2026).



## ПРИЛОЖЕНИЕ А

### Исходный код программы

#### ПРИЛОЖЕНИЕ А.1

##### Точка входа и графический интерфейс

```
import tkinter as tk
from pathlib import Path
from tkinter import messagebox, ttk

from box_geometry import BoxParams, build_panels, fmt,
segment_bounds, verify_assembly
from dxf_writer import save_dxf_file

APP_TITLE = "ElectronicsBox – генератор DXF"

class GeneratorApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title(APP_TITLE)
        self.geometry("560x420")
        self.resizable(False, False)
        self.length_var = tk.StringVar(value="100.0")
        self.depth_var = tk.StringVar(value="100.0")
        self.height_var = tk.StringVar(value="100.0")
        self.thickness_var = tk.StringVar(value="3.0")
        self.finger_var = tk.StringVar(value="6.0")
        self.file_var = tk.StringVar(value="electronics_box.dxf")
        self.status = tk.StringVar(value="")
        self.make_window()

    def make_window(self):
        main = ttk.Frame(self, padding=24)
        main.pack(fill=tk.BOTH, expand=True)
        main.columnconfigure(1, weight=1)

        ttk.Label(main, text="Коробка ElectronicsBox",
font=("TkDefaultFont", 16, "bold")).grid(row=0, column=0,
columnspan=2, pady=(0, 18))
```

```

rows = [
    ("Длина (внутренняя ширина), мм", self.length_var),
    ("Глубина (внутренняя глубина), мм", self.depth_var),
    ("Высота (внутренняя высота), мм", self.height_var),
    ("Толщина материала, мм", self.thickness_var),
    ("Ширина шипа, мм", self.finger_var),
    ("Имя выходного файла", self.file_var),
]

for row, data in enumerate(rows, start=1):
    title, variable = data
    ttk.Label(main, text=title).grid(row=row, column=0,
sticky="w", pady=5, padx=(0, 18))
    ttk.Entry(main, textvariable=variable).grid(row=row,
column=1, sticky="ew", pady=5)

buttons = ttk.Frame(main)
buttons.grid(row=7, column=0, colspan=2, sticky="ew",
pady=(18, 8))
buttons.columnconfigure(0, weight=1)

ttk.Button(buttons, text="Сгенерировать DXF",
command=self.generate_dxf).grid(row=0, column=0, sticky="ew")
ttk.Label(main, textvariable=self.status,
foreground="#245070").grid(row=8, column=0, colspan=2, sticky="w")

def get_params(self):
    return BoxParams(
        self.read_number(self.length_var.get(), "Длина"),
        self.read_number(self.depth_var.get(), "Глубина"),
        self.read_number(self.height_var.get(), "Высота"),
        self.read_number(self.thickness_var.get(), "Толщина
материала"),
        self.read_number(self.finger_var.get(), "Ширина шипа"),
    )

def read_number(self, text, name):
    try:
        return float(text.replace(",", "."))
    except ValueError:
        raise ValueError(f"{name} указана неправильно")

def output_path(self, extension):

```

```

name = self.file_var.get().strip()
if not name:
    name = "electronics_box.dxf"

path = Path(name)
if path.suffix.lower() != extension:
    path = path.with_suffix(extension)
if not path.is_absolute():
    path = Path("output") / path
return path

def generate_dxf(self):
    try:
        params = self.get_params()
        verify_assembly(params)
        path = self.output_path(".dxf")
        save_dxf_file(params, path)
        self.show_result(params, path)
    except Exception as error:
        messagebox.showerror("Ошибка", str(error))

def show_result(self, params, path):
    segments = []
    for panel in build_panels(params):
        segments.extend(panel.segments)
    min_x, min_y, max_x, max_y = segment_bounds(segments)
    text = f"Готово: {path} ({fmt(max_x - min_x)} x {fmt(max_y -
min_y)} мм)"
    self.status.set(text)
    messagebox.showinfo("Готово", text)

def main():
    app = GeneratorApp()
    app.mainloop()

if __name__ == "__main__":
    main()

```

Листинг А.1 — Файл laser\_box\_generator.py

## ПРИЛОЖЕНИЕ А.2

### Расчёт геометрии

```
import math
from dataclasses import dataclass

Point = tuple[float, float]

CUT = "CUT"
HOLES = "HOLES"

EPSILON = 0.0001
ZERO_EPSILON = 0.000001

CIRCLE_STEPS = 176
CORNER_STEPS = 48

@dataclass
class BoxParams:
    length: float = 100
    depth: float = 100
    height: float = 100
    thickness: float = 3
    finger_width: float = 6
    sheet_width: float = 300
    sheet_height: float = 460

    def __post_init__(self):
        self.length = float(self.length)
        self.depth = float(self.depth)
        self.height = float(self.height)
        self.thickness = float(self.thickness)
        self.finger_width = float(self.finger_width)
        self.sheet_width = float(self.sheet_width)
        self.sheet_height = float(self.sheet_height)

@dataclass(frozen=True)
class Segment:
    start: Point
    end: Point
```

```

layer: str = CUT

@property
def values(self):
    x1, y1 = self.start
    x2, y2 = self.end
    return x1, y1, x2, y2

@dataclass(frozen=True)
class Panel:
    title: str
    x: float
    y: float
    segments: list[Segment]

def fmt(value):
    return f"{value:.1f}".rstrip("0").rstrip(".")

def validate_params(params):
    values = [
        ("Длина", params.length),
        ("Глубина", params.depth),
        ("Высота", params.height),
        ("Толщина материала", params.thickness),
        ("Ширина шипа", params.finger_width),
        ("Ширина листа", params.sheet_width),
        ("Высота листа", params.sheet_height),
    ]

    for name, value in values:
        if not math.isfinite(value):
            raise ValueError(f"{name} указана неправильно")

    for name, value in values:
        if value <= 0:
            raise ValueError(f"{name} должна быть положительным числом")

    if params.thickness < 2 or params.thickness > 6:
        raise ValueError("Толщина материала должна быть от 2 до 6")

```

```

мм")

    if params.finger_width < params.thickness or params.finger_width
> 20:
        raise ValueError("Ширина шипа должна быть от толщины
материала до 20 мм")

    min_side = max(45, params.thickness * 12, params.finger_width *
5)
    min_height = max(35, params.thickness * 10, params.finger_width *
5)

    if params.length < min_side:
        raise ValueError(f"Длина должна быть не меньше
{fmt(min_side)} мм")
    if params.depth < min_side:
        raise ValueError(f"Глубина должна быть не меньше
{fmt(min_side)} мм")
    if params.height < min_height:
        raise ValueError(f"Высота должна быть не меньше
{fmt(min_height)} мм")

def verify_assembly(params):
    validate_params(params)

    panels = build_panels(params)
    segments = collect_segments(panels)
    bounds = segment_bounds(segments)
    if bounds is None:
        raise ValueError("В чертеже нет линий")

    min_x, min_y, max_x, max_y = bounds
    if min_x < -0.001 or min_y < -0.001:
        raise ValueError("Чертеж выходит в отрицательные координаты")

    layout_width = max_x - min_x
    layout_height = max_y - min_y

    if layout_width > params.sheet_width + 0.001:
        raise ValueError(f"Раскладка шире листа: {fmt(layout_width)}
мм > {fmt(params.sheet_width)} мм")
    if layout_height > params.sheet_height + 0.001:

```

```
        raise ValueError(f"Раскладка выше листа: {fmt(layout_height)}  
мм > {fmt(params.sheet_height)} мм")
```

```
def build_panels(params):  
    validate_params(params)  
    return build_generated_panels(params)
```

```
def collect_segments(panels):  
    segments = []  
    for panel in panels:  
        segments.extend(panel.segments)  
    return segments
```

```
# Единый параметрический генератор.
```

```
def build_generated_panels(params):  
    contours = []  
  
    for x, y, width, style in wall_layout(params):  
        contours.extend(wall_contours(x, y, width, params.height,  
params, style))  
  
    bottom_x, bottom_y, bottom_width, bottom_height =  
bottom_layout(params)  
    contours.extend(bottom_contours(bottom_x, bottom_y, bottom_width,  
bottom_height, params))  
  
    top_x, top_y, top_width, top_height = top_layout(params)  
    contours.extend(top_contours(top_x, top_y, top_width, top_height,  
params))  
  
    mounts_x, mounts_y = mounts_layout(params)  
    contours.extend(mounts_contours(mounts_x, mounts_y, params))  
  
    return [make_panel("ElectronicsBox", 0, 0, contours)]  
  
def make_panel(title, x, y, contours):  
    segments = contours_to_segments(contours)  
    return Panel(title, x, y, segments)
```

```

def layout_margin():
    return 10.0

def layout_gap(params):
    return params.thickness / 2

def wall_layout(params):
    margin = layout_margin()
    gap = layout_gap(params)
    x1 = margin
    x2 = margin + params.length + gap
    y1 = margin
    y2 = margin + params.height + gap

    return [
        (x1, y1, params.length, "outer"),
        (x2, y1, params.depth, "inner"),
        (x2, y2, params.depth, "inner"),
        (x1, y2, params.length, "outer"),
    ]

def bottom_layout(params):
    margin = layout_margin()
    gap = layout_gap(params)
    y = margin + params.height * 2 + gap * 2
    return margin, y, bottom_plate_width(params), params.depth

def top_layout(params):
    bottom_x, bottom_y, _, bottom_height = bottom_layout(params)
    y = bottom_y + bottom_height + layout_gap(params)
    width = max(params.length - 2 * params.thickness,
params.thickness)
    height = max(params.depth - 2 * params.thickness,
params.thickness)
    return bottom_x, y, width, height

def mounts_layout(params):

```



```

top_x, top_y, top_width, _ = top_layout(params)
return top_x + top_width + layout_gap(params), top_y

def wall_contours(x, y, width, height, params, style):
    slot_offset = wall_mount_slot_offset(params)
    slot_y = y + height - slot_offset - params.thickness
    return [
        (
            HOLES,
            rect_contour(
                x + slot_offset,
                slot_y,
                params.finger_width,
                params.thickness,
            ),
        ),
        (
            HOLES,
            rect_contour(
                x + width - slot_offset - params.finger_width,
                slot_y,
                params.finger_width,
                params.thickness,
            ),
        ),
        (CUT, wall_outline(x, y, width, height, params, style)),
    ]

def wall_outline(x, y, width, height, params, style):
    thickness = params.thickness
    top_start = top_finger_start(width, params)
    points = [(x + thickness, y + thickness), (x + top_start, y +
thickness)]
    add_wall_top_fingers(points, x, y, width, params)

    if style == "outer":
        points.append((x + width, y + thickness))
        points.extend(wall_side_down(x + width, x + width -
thickness, y, height, params))
        points.extend([(x + thickness, y + height), (x, y + height)])
        points.extend(wall_side_up(x, x + thickness, y, height,

```

```

params))
    else:
        points.extend(wall_side_down(x + width - thickness, x +
width, y, height, params))
        points.append((x + thickness, y + height))
        points.extend(wall_side_up(x + thickness, x, y, height,
params))

    return points

def add_wall_top_fingers(points, x, y, width, params):
    thickness = params.thickness
    finger_width = params.finger_width
    start = top_finger_start(width, params)

    for index in range(joint_count(width, params)):
        current_x = start + index * finger_width * 2
        points.extend(
            [
                (x + current_x, y),
                (x + current_x + finger_width, y),
                (x + current_x + finger_width, y + thickness),
                (x + current_x + finger_width * 2, y + thickness),
            ]
        )

    points[-1] = (x + width - thickness, y + thickness)

def wall_side_down(edge_x, notch_x, y, height, params):
    points = []
    for offset in side_finger_offsets(height, params):
        points.extend(
            [
                (edge_x, y + offset),
                (notch_x, y + offset),
                (notch_x, y + offset + params.finger_width),
                (edge_x, y + offset + params.finger_width),
            ]
        )

    points.append((edge_x, y + height))

```

```

        if edge_x > notch_x and abs(edge_x - notch_x) ==
params.thickness:
            points.append((notch_x, y + height))

    return points

def wall_side_up(edge_x, notch_x, y, height, params):
    points = []
    offsets = side_finger_offsets(height, params)

    for offset in reversed(offsets):
        upper_offset = offset + params.finger_width
        points.extend(
            [
                (edge_x, y + upper_offset),
                (notch_x, y + upper_offset),
                (notch_x, y + offset),
                (edge_x, y + offset),
            ]
        )

    if edge_x < notch_x:
        points.extend([(edge_x, y + params.thickness), (notch_x, y +
params.thickness)])
    else:
        points.append((edge_x, y + params.thickness))

    return points

def joint_count(length, params):
    available = max(0.0, length - 2 * params.thickness)
    return max(1, int(available // (params.finger_width * 2)))

def top_finger_start(length, params):
    available = max(0.0, length - 2 * params.thickness)
    used = joint_count(length, params) * params.finger_width * 2
    return params.thickness + (available - used) / 2 +
params.thickness

```

```

def side_finger_offsets(length, params):
    start = top_finger_start(length, params) + params.thickness / 2
    return [start + index * params.finger_width * 2 for index in
range(joint_count(length, params))]

def bottom_contours(x, y, width, height, params):
    radius = bottom_corner_radius(params)
    contours = [
        (HOLES, circle_contour(x + radius, y + radius,
params.thickness / 2, 0.0)),
        (CUT, rounded_bottom_outline(x, y, width, height, params)),
        (HOLES, circle_contour(x + width - radius, y + radius,
params.thickness / 2, 90.0)),
    ]

    side = bottom_side_extension(params)
    for slot_y in bottom_slot_ys(y, height, params):
        contours.append(
            (
                HOLES,
                rect_contour(
                    x + width - side - params.thickness,
                    slot_y,
                    params.thickness,
                    params.finger_width,
                ),
            )
        )

    contours.append(
        (HOLES, circle_contour(x + width - radius, y + height -
radius, params.thickness / 2, 180.0))
    )
    contours.append((HOLES, circle_contour(x + radius, y + height -
radius, params.thickness / 2, -90.0)))

    for slot_y in reversed(bottom_slot_ys(y, height, params)):
        contours.append((HOLES, rect_contour(x + side, slot_y,
params.thickness, params.finger_width)))

    return contours

```

```

def rounded_bottom_outline(x, y, width, height, params):
    thickness = params.thickness
    finger_width = params.finger_width
    radius = bottom_corner_radius(params)
    side = bottom_side_extension(params)
    start = top_finger_start(params.length, params)
    count = joint_count(params.length, params)

    points = [(x + radius, y), (x + side + thickness, y), (x + side +
start, y)]

    for index in range(count):
        current_x = x + side + start + index * finger_width * 2
        points.extend(
            [
                (current_x, y + thickness),
                (current_x + finger_width, y + thickness),
                (current_x + finger_width, y),
                (current_x + finger_width * 2, y),
            ]
        )

    points[-1] = (x + width - side - thickness, y)
    points.append((x + width - radius, y))
    points.extend(arc_points(x + width - radius, y + radius, radius,
-90.0, 0.0)[1:])
    points.append((x + width, y + height - radius))
    points.extend(arc_points(x + width - radius, y + height - radius,
radius, 0.0, 90.0)[1:])
    points.extend([(x + width - side - thickness, y + height), (x +
width - side - start, y + height)])

    for index in range(count):
        current_x = x + width - side - start - index * finger_width *
2
        points.extend(
            [
                (current_x, y + height - thickness),
                (current_x - finger_width, y + height - thickness),
                (current_x - finger_width, y + height),
                (current_x - finger_width * 2, y + height),
            ]
        )

```

```

    )

    points[-1] = (x + side + thickness, y + height)
    points.append((x + radius, y + height))
    points.extend(arc_points(x + radius, y + height - radius, radius,
90.0, 180.0)[1:])
    points.append((x, y + radius))
    points.extend(arc_points(x + radius, y + radius, radius, 180.0,
270.0)[1:])
    return points

def bottom_plate_width(params):
    return params.length + 2 * bottom_side_extension(params)

def bottom_side_extension(params):
    return max(14.0, params.thickness * 4 + 2)

def bottom_corner_radius(params):
    return max(7.0, params.thickness * 2 + 1)

def bottom_slot_ys(y, height, params):
    start = top_finger_start(height, params)
    return [y + start + index * params.finger_width * 2 for index in
range(joint_count(height, params))]

def top_contours(x, y, width, height, params):
    offset = min(params.finger_width + params.thickness * 0.75,
width / 2, height / 2)
    radius = params.thickness / 2
    return [
        (HOLES, circle_contour(round(x + offset, 3), round(y +
offset, 3), radius, 0.0)),
        (CUT, outline_rect_contour(x, y, width, height)),
        (HOLES, circle_contour(round(x + width - offset, 3), round(y
+ offset, 3), radius, 90.0)),
        (HOLES, circle_contour(round(x + width - offset, 3), round(y
+ height - offset, 3), radius, 180.0)),
        (HOLES, circle_contour(round(x + offset, 3), round(y + height

```

```

- offset, 3), radius, -90.0)),
    ]

def mounts_contours(x, y, params):
    size = mount_size(params)
    radius = max(1.0, params.thickness / 3)
    gap = params.thickness
    second_pair_y = y + size + params.thickness + gap

    upper_x = x + size / 4
    lower_x = x

    upper_y_1 = y
    lower_y_1 = y + params.thickness
    upper_y_2 = second_pair_y
    lower_y_2 = second_pair_y + params.thickness

    return [
        (CUT, upper_mount_contour(upper_x, upper_y_1, size, params)),
        (HOLES, circle_contour(upper_x + size * 0.6, upper_y_1 + size
* 0.4, radius, 90.0)),
        (CUT, lower_mount_contour(lower_x, lower_y_1, size, params)),
        (HOLES, circle_contour(lower_x + size * 0.4, lower_y_1 + size
* 0.6, radius, -90.0)),
        (CUT, upper_mount_contour(upper_x, upper_y_2, size, params)),
        (HOLES, circle_contour(upper_x + size * 0.6, upper_y_2 + size
* 0.4, radius, 90.0)),
        (CUT, lower_mount_contour(lower_x, lower_y_2, size, params)),
        (HOLES, circle_contour(lower_x + size * 0.4, lower_y_2 + size
* 0.6, radius, -90.0)),
    ]

def mount_size(params):
    nominal_size = 28.0
    scale = min(params.length, params.depth, params.height) / 100.0
    minimum_size = params.finger_width + params.thickness * 2
    return max(minimum_size, nominal_size * scale)

def wall_mount_slot_offset(params):
    return mount_notch_center(mount_size(params), params)

```

```

def mount_notch_center(size, params):
    return size / 2 - params.thickness / 2

def upper_mount_contour(x, y, size, params):
    thickness = params.thickness
    finger_width = params.finger_width
    notch_center = mount_notch_center(size, params)
    notch_start = notch_center - finger_width / 2
    notch_end = notch_center + finger_width / 2
    inner_edge = size - thickness

    return [
        (x, y + thickness),
        (x + notch_start, y + thickness),
        (x + notch_start, y),
        (x + notch_end, y),
        (x + notch_end, y + thickness),
        (x + inner_edge, y + thickness),
        (x + inner_edge, y + notch_center),
        (x + size, y + notch_center),
        (x + size, y + notch_center + finger_width),
        (x + inner_edge, y + notch_center + finger_width),
        (x + inner_edge, y + size),
        (x, y + thickness),
    ]

def lower_mount_contour(x, y, size, params):
    thickness = params.thickness
    finger_width = params.finger_width
    notch_center = mount_notch_center(size, params)
    notch_start = notch_center - finger_width / 2
    notch_end = notch_center + finger_width / 2
    inner_edge = size - thickness

    return [
        (x + size, y + inner_edge),
        (x + notch_center + finger_width, y + inner_edge),
        (x + notch_center + finger_width, y + size),
        (x + notch_center, y + size),
    ]

```



```

        (x + notch_center, y + inner_edge),
        (x + thickness, y + inner_edge),
        (x + thickness, y + notch_end),
        (x, y + notch_end),
        (x, y + notch_start),
        (x + thickness, y + notch_start),
        (x + thickness, y),
        (x + size, y + inner_edge),
    ]

```

```

def rect_contour(x, y, width, height):
    return [
        (x, y + height),
        (x + width, y + height),
        (x + width, y),
        (x, y),
        (x, y + height),
    ]

```

```

def outline_rect_contour(x, y, width, height):
    return [
        (x, y),
        (x + width, y),
        (x + width, y + height),
        (x, y + height),
        (x, y),
    ]

```

```

def circle_contour(cx, cy, radius, start_degrees):
    points = []
    for index in range(CIRCLE_STEPS + 1):
        angle = math.radians(start_degrees - 360.0 * index /
CIRCLE_STEPS)
        points.append((cx + radius * math.cos(angle), cy + radius *
math.sin(angle)))
    return points

```

```

def arc_points(cx, cy, radius, start_degrees, end_degrees):
    points = []

```

```

        for index in range(CORNER_STEPS + 1):
            angle = math.radians(start_degrees + (end_degrees -
start_degrees) * index / CORNER_STEPS)
            points.append((cx + radius * math.cos(angle), cy + radius *
math.sin(angle)))
        return points

def contours_to_segments(contours):
    segments = []
    for layer, contour in contours:
        segments.extend(contour_to_segments(contour, layer))
    return segments

def contour_to_segments(contour, layer):
    if len(contour) < 2:
        return []

    points = []
    for point in contour:
        points.append(clean_point(point))

    if points[0] != points[-1]:
        points.append(points[0])

    segments = []
    for start, end in zip(points, points[1:]):
        length = math.hypot(end[0] - start[0], end[1] - start[1])
        if length >= EPSILON:
            segments.append(Segment(start, end, layer))

    return segments

def clean_point(point):
    x, y = point
    return clean_number(x), clean_number(y)

def clean_number(value):
    rounded = round(value, 6)
    if abs(rounded) < ZERO_EPSILON:

```

```

        return 0.0
    return rounded

def segment_bounds(segments):
    if not segments:
        return None

    min_x = min_y = math.inf
    max_x = max_y = -math.inf

    for segment in segments:
        for x, y in [segment.start, segment.end]:
            min_x = min(min_x, x)
            min_y = min(min_y, y)
            max_x = max(max_x, x)
            max_y = max(max_y, y)

    return min_x, min_y, max_x, max_y

```

Листинг А.2 — Файл box\_geometry.py

## ПРИЛОЖЕНИЕ А.3

### Экспорт DXF

```

import ezdxf

from box_geometry import build_panels, segment_bounds,
verify_assembly

LAYER_COLORS = {"CUT": 7, "HOLES": 5}
LINE_STYLES = {
    "CUT": {"color": 250, "linetype": "K5LT32768", "lineweight": 18},
    "HOLES": {"color": 5, "linetype": "K5LT32769", "lineweight": 18},
}

def create_dxf_document(params):
    verify_assembly(params)
    segments = [segment for panel in build_panels(params) for segment
in panel.segments]

```

```

doc = ezdxf.new("R2018")
doc.header["$MEASUREMENT"] = 1
doc.header["$INSUNITS"] = 4
doc.units = 4

add_layers(doc)
msp = doc.modelspace()
for segment in segments:
    x1, y1 = segment.start
    x2, y2 = segment.end
    msp.add_line((x1, y1, 0), (x2, y2, 0),
dxfattribs=line_attribs(segment.layer))

bounds = segment_bounds(segments)
if bounds is not None:
    min_x, min_y, max_x, max_y = bounds
    doc.header["$EXTMIN"] = (min_x, min_y, 0)
    doc.header["$EXTMAX"] = (max_x, max_y, 0)

return doc

def add_layers(doc):
    for linetype in {style["linetype"] for style in
LINE_STYLES.values()}:
        if not doc.linetypes.has_entry(linetype):
            doc.linetypes.add(linetype, pattern=[0.0],
description="")

    for layer, color in LAYER_COLORS.items():
        if not doc.layers.has_entry(layer):
            doc.layers.new(name=layer, dxfattribs={"color": color})

def line_attribs(layer):
    attrs = {"layer": layer}
    attrs.update(LINE_STYLES.get(layer, {}))
    return attrs

def save_dxf_file(params, path):
    doc = create_dxf_document(params)

```

```
path.parent.mkdir(parents=True, exist_ok=True)  
doc.saveas(path)
```

Листинг A.3 — Файл dxf\_writer.py